Universidad de Buenos Aires
Facultad de Ingeniería

Paradigmas de Programación
Trabajo Práctico Nº 2

Integrantes:

Alexander Coronado Nurhayati	111256
Aylén Bartomeo	111738
Ignacio Berango	111453
Mailen Melina Gonzalez	111893

2do Cuatrimestre de 2025

Índice

1. Introducción **2**

2. Cómo compilar y ejecutar el programa **3**

3. Funcionalidades implementadas **5**

4. Cómo se juega al juego **8**

5. Diseño del sistema **11**

5.1. Arquitectura general 11

5.2. Diagrama de clases UML 11

5.2.1. UML MAIN 12

5.2.2. UML ARCHIVOS 13

5.2.3. UML MODELO 14

5.2.4. UML VISTA 15

5.2.5. UML CONTROLADOR 15

5.3. Diagrama de secuencia 17

6. Hipótesis **19**

7. Conclusiones **22**

1. Introducción

El presente informe describe el desarrollo del **Trabajo Práctico N°2** de la materia *Paradigmas de Programación*, cuyo objetivo fue diseñar e implementar un sistema de combate táctico por turnos denominado *Class Emblem*. El trabajo se desarrolló íntegramente en **Java 21**, aplicando principios de programación orientada a objetos y separación clara de responsabilidades.

El sistema modela un enfrentamiento estratégico entre dos reinos *Reino Druida* y el *Reino Nigromántico* sobre un tablero cargado desde archivos externos. El juego incorpora una amplia variedad de elementos del dominio: terrenos con efectos especiales, unidades con atributos y estados internos, equipamiento ofensivo y de soporte, acciones de combate y curación, preparación de emboscadas, movimiento restringido por tipos de casillas, despliegue inicial y decisiones tácticas gestionadas por turnos alternados.

El **modelo** implementa la lógica central del juego: representación del tablero, registro y creación de casillas, cálculo de casillas alcanzables, gestión de unidades (vida, ataque, defensa, magia, ocultamiento, movimiento disponible, equipamiento), reglas de despliegue, combate físico y mágico, curación mediante báculos, consumo de equipamiento, sistema de emboscadas, condiciones de victoria y rendición.

El componente de **controladores** coordina el flujo completo de la partida en tres etapas principales: (1) inicio del juego mediante la carga de mapa y ejército, posicionamiento de los Lords y despliegue inicial; (2) gestión del turno mediante menús de acciones, movimiento, ataque/curación, emboscada e información contextual; y (3) un submenú dedicado a la inspección detallada de las unidades, tanto en el tablero como en la reserva.

Finalmente, la **vista por consola** presenta la información del tablero, unidades, opciones y estado general de la partida utilizando renderización textual, listas formateadas y códigos de color, brindando una experiencia interactiva clara y consistente con el modelo.

Este informe detalla la arquitectura desarrollada, las decisiones de diseño adoptadas, los diagramas que representan la estructura del sistema, las hipótesis necesarias para resolver ambigüedades del enunciado y el funcionamiento completo del programa, desde la carga inicial hasta la resolución de la partida.

2. Cómo compilar y ejecutar el programa

Requisitos

Para compilar y ejecutar el proyecto se necesita:

- Java 21 (cualquier distribución)
- Apache Maven 3.8+
- Consola / terminal

Instalación

Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install -y openjdk-21-jdk maven
```

macOS (Homebrew)

```
brew install openjdk@21
brew install maven
```

Si es necesario, agregar Java al PATH:

```
export PATH="/usr/local/opt/openjdk@21/bin:$PATH"
```

Windows (winget)

```
winget install EclipseAdoptium.Temurin.21.JDK
winget install Apache.Maven
```

Verificación

```
java -version
mvn -version
```

Ambos deberían mostrar Java 21 y Maven instalado correctamente.

Cómo compilar y ejecutar el juego

El TP debe correrse exclusivamente mediante Maven.

Compilar

```
mvn compile
```

Ejecutar

```
mvn exec:java
```

Este comando ya ejecuta la main class porque definimos `<exec.mainClass>org.main.Main</exec.mainClass>` en el `pom.xml`.

Compilar y ejecutar

```
mvn compile exec:java
```

Este comando compila el código, ejecuta la clase main y levanta automáticamente los recursos desde `src/main/resources`.

3. Funcionalidades implementadas

El sistema desarrollado implementa un juego táctico por turnos completamente funcional, siguiendo y ampliando las especificaciones del enunciado del Trabajo Práctico. Las funcionalidades implementadas cubren todos los aspectos necesarios para desarrollar una partida completa: desde la carga de recursos externos hasta la resolución del combate, la administración de turnos y la interacción con el jugador a través de una interfaz textual. A continuación se describen las características más relevantes del sistema, organizadas según las etapas y módulos principales del juego.

Carga de archivos y configuración inicial

- Lectura y procesamiento de archivos CSV de **mapas** y **ejércitos**, con validación de estructura, formato y contenido. Se descartaron filas inválidas o encabezados no estándar para asegurar consistencia en la información cargada.
- Construcción dinámica del tablero a partir del mapa leído: cada símbolo del archivo corresponde a un tipo de casilla, instanciada mediante una fábrica especializada (`FabricaCasillas`). Este proceso garantiza extensibilidad para agregar nuevos tipos de terreno.
- Generación de unidades de ambos bandos (Lord y tropas), con asignación automática de atributos: puntos de vida, ataque, defensa, magia, movimiento y equipamiento.
- Selección interactiva del archivo de mapa y del archivo de ejército a través de la consola, facilitando la reutilización del programa con distintos escenarios de juego.
- Ubicación manual del **Lord** de cada jugador en casillas transitables y no ocupadas, permitiendo mayor libertad estratégica en la preparación inicial.
- Implementación completa de la **fase de despliegue**, en la cual las unidades restantes se colocan alrededor del Lord respetando reglas de adyacencia, ocupación y tipo de terreno. Cuando el terreno lo permite, se habilita la opción de desplegar en modo *oculto*.

Modelo del tablero y terrenos

- Representación del tablero como una matriz de casillas, cada una con tipo, posición, características de transitabilidad y efectos especiales.
- Implementación de múltiples terrenos con comportamientos específicos:
  - **Castillo**: curación al finalizar el turno y aumento de defensa.
  - **Pantano**: penalización al movimiento.
  - **Bosque**: habilita acciones de emboscada, aumenta ataque y magia.
  - **Agua / Acantilado / Enredadera**: intransitable para todas las unidades.
  - **Área contaminada**: daño continuo mientras la unidad permanezca allí.

- Cálculo dinámico de casillas alcanzables según el movimiento restante de cada unidad. Se muestran visualmente al jugador en el tablero para facilitar la toma de decisiones.
- Verificación automática de casillas ocupadas e intransitables, garantizando que los movimientos realizados sean legales dentro de las reglas del juego.
- Inclusión de efectos *al entrar* y *al finalizar el turno*, integrados en el flujo del sistema mediante interfaces de comportamiento.

Unidades, atributos y estados

- Gestión completa del ciclo de vida de cada unidad: creación, vida, muerte, posición en tablero o reserva, visibilidad (oculto/visible), equipamiento y regeneración de acciones por turno.
- Movimiento en ocho direcciones con validación estricta de distancia, camino permitido y consumo progresivo de puntos de movimiento en función del terreno.
- Estados específicos de turno:
  - Unidades que no actúan reciben un **bonus defensivo** para el turno rival.
  - Restauración automática del movimiento al inicio del turno propio.
  - Reseteo de bonus temporales al finalizar ciertos ciclos del turno.
- Equipamiento variado con efectos diferenciados:
  - **Armas**: ataque físico o mágico según tipo.
  - **Grimorios**: ataques mágicos de rango mayor.
  - **Báculos**: curación/defensa de aliados mediante estrategias programadas.
- Implementación del sistema de **emboscada**, en el cual las unidades pueden permanecer ocultas y atacar desde sigilo dependiendo del terreno. Incluye reglas para revelación automática.

Sistema de combate y acciones

- Acción de **mover**: lista interactiva de unidades que pueden desplazarse, renderizado del tablero con casillas disponibles marcadas y validación del destino.
- Acción de **atacar/curar**: detección del tipo de equipamiento para decidir la acción, validación del rango y resolución del efecto (daño o curación). Se consideran estadísticas, equipamiento y efectos del terreno.
- Cálculo determinista del daño físico y mágico, evitando aleatoriedad y facilitando la trazabilidad de los resultados en consola.
- Curación y soporte mediante báculos, implementados con estrategias específicas que encapsulan efectos de curación, aumento de magia u otros.
- Eliminación automática de unidades derrotadas, con liberación inmediata de la casilla para permitir nuevas acciones del oponente.
- Posibilidad de **desplegar unidades** durante la partida si quedan tropas en reserva.

- Acción de **emboscada**: preparación y activación del estado de ocultamiento, si la casilla lo permite.
- Acción de **rendirse**: finalización inmediata de la partida declarando ganador al bando contrario.

Flujo de turnos y finalización del juego

- Alternancia automática de turnos entre el **Reino Druida** y el **Reino Nigromántico**, coordinada por el controlador principal.
- Actualización del estado global al inicio y fin de cada turno: restauración de movimientos, aplicación de efectos de terreno, limpieza de bonos temporales, y preparación de las unidades del bando activo.
- Menú completo de acciones que centraliza todas las posibilidades del turno: mover, actuar, desplegar, emboscar, ver el estado de las unidades, obtener información del mapa, finalizar turno y rendirse.
- Detección de condiciones de finalización:
  - muerte del Lord,
  - rendición voluntaria del jugador.

Interfaz por consola

- Renderización textual del tablero con símbolos y colores ANSI, destacando tipos de terreno y bandos.
- Presentación de menús diferenciados según la fase del juego: inicio, turno principal y menú de unidades.
- Listas formateadas con información completa de cada unidad: nombre, posición, puntos de vida, equipamiento y estado.
- Interacción simple e intuitiva basada en selección numérica, asegurando accesibilidad y claridad incluso sin interfaz gráfica.

4. Cómo se juega al juego

El juego se desarrolla íntegramente desde la consola, mediante la interacción con menús numéricos. A través de estos menús, los jugadores toman todas las decisiones de manera secuencial, siguiendo un flujo que abarca desde la carga inicial de archivos hasta la resolución de los combates. A continuación se describe en detalle el ciclo completo de jugabilidad y la forma en que se ejecutan las acciones principales del juego.

Pantalla de inicio y selección de archivos

Al iniciar el programa, se despliega una pantalla de bienvenida que solicita al usuario elegir los recursos que definirán la partida. Por consola se muestran los archivos disponibles dentro de los recursos del proyecto, entre ellos los mapas y los ejércitos. El jugador debe seleccionar, mediante ingreso numérico, las siguientes opciones:

- el **archivo de mapa** sobre el cual se desarrollará la partida;
- el **archivo de ejército** correspondiente a ambos bandos.

Cada mapa define el tipo de terreno de cada casilla (bosque, pantano, castillo, etc.), mientras que el archivo de ejército determina qué unidades y equipamientos estarán disponibles para ambos jugadores. Una vez cargados, el sistema construye el tablero y crea las instancias de las unidades con sus estadísticas, armas y habilidades iniciales. De esta manera, se garantiza una configuración completamente reproducible de la partida.

Posicionamiento de los Lords

Cada bando comienza la partida posicionando manualmente a su **Lord** en el tablero, que representa la unidad principal y la condición de victoria o derrota. El sistema solicita coordenadas válidas (fila, columna) y valida que la casilla elegida sea transitable y no esté ocupada. Si el jugador elige una posición inválida, se notifica el error y se vuelve a pedir la entrada, evitando configuraciones imposibles.

Fase de despliegue

Finalizada la colocación de los Lords, cada jugador despliega sus unidades restantes una por una. El sistema guía al jugador mostrando el estado actual del tablero y listando las unidades que aún se encuentran en reserva. Al igual que con los Lords, si el jugador elige una posición inválida, se pide que lo reintente.

- Muestra las unidades en reserva del bando actual.
- Se elige la unidad a desplegar por su índice numérico.
- Se ingresan coordenadas adyacentes al Lord propio, siempre en casillas transitables y libres.

- Si el terreno lo permite, el juego pregunta si se desea desplegar en modo **oculto**, lo que activa una mecánica de emboscada.

Esta fase permite a cada jugador definir su formación inicial, equilibrando la ofensiva, la defensa y el posicionamiento táctico según el terreno. La fase termina cuando el jugador decide no desplegar más unidades, continuando luego con el despliegue del bando rival.

Comienzo del combate

Finalizado el despliegue para ambos bandos, comienza el combate por turnos. El sistema muestra en cada turno:

- el número de turno actual;
- el tablero renderizado con colores que distinguen ambos bandos;
- la lista de todas las unidades en el tablero, indicando su nombre, posición, puntos de vida, y si pueden moverse o actuar en ese turno.

El diseño busca ofrecer al jugador toda la información relevante para planificar su siguiente acción, sin necesidad de revisar datos manualmente.

Menú de acciones del turno

Durante su turno, el jugador actual puede seleccionar una de las siguientes acciones:

- **1. Mover unidad:** se listan las unidades que aún pueden desplazarse. Al seleccionar una, el sistema muestra las casillas alcanzables según su rango de movimiento y el tipo de terreno. El jugador elige la coordenada destino y, si es válida, la unidad se mueve, consumiendo su movimiento disponible.
- **2. Atacar o curar:** el jugador selecciona una unidad capaz de actuar. Si la unidad posee un arma o grimorio, puede atacar a enemigos dentro de su rango. Si empuña un báculo, puede curar a aliados. Tras la acción, el equipamiento consume un uso y la unidad no puede volver a actuar ese turno.
- **3. Ver unidades:** abre un submenú que permite listar las unidades desplegadas o en reserva, y consultar en detalle las estadísticas de una unidad específica (puntos de vida, defensa, magia, equipamiento, etc.).
- **4. Desplegar unidades restantes:** las unidades que aún estén en reserva pueden incorporarse al campo de batalla, siempre que se ubiquen adyacentes al Lord.
- **5. Preparar emboscada:** una unidad situada en un terreno apto (como bosque) puede ocultarse y esperar para atacar más adelante. Este modo permite tácticas de sorpresa y defensa.
- **6. Información del mapa:** muestra la descripción completa de los terrenos disponibles y los efectos que aplican a las unidades que los ocupan.

- **7. Terminar turno:** el jugador finaliza voluntariamente su turno y activa el del bando contrario. Antes de ceder el control, se aplican los efectos de terreno y bonificaciones de fin de turno.
- **8. Rendirse:** el jugador puede declarar su derrota voluntariamente. El sistema imprime un mensaje de victoria para el bando opuesto y concluye la partida.

Reglas adicionales de jugabilidad

- Cada unidad puede moverse y actuar solo una vez por turno.
- Si una unidad no realiza ninguna acción, obtiene un bonus de defensa temporal durante el próximo turno del enemigo.
- Las unidades derrotadas desaparecen del tablero, liberando su casilla.
- Los usos del equipamiento son limitados y pueden agotarse con el tiempo.
- Las acciones se ejecutan de forma secuencial y no pueden cancelarse una vez confirmadas.

Condiciones de finalización

- Derrota del Lord propio.
- Rendición voluntaria.

En cualquiera de estos casos, el sistema declara automáticamente al ganador y finaliza la partida.

5. Diseño del sistema

5.1. Arquitectura general

El diseño sigue arquitectura MVC, garantizando bajo acoplamiento y separación de responsabilidades.

Modelo (org.modelo)

- Lógica central del juego.
- Unidades, equipamientos, terrenos, tablero, turnos.
- Reglas de movimiento, daño, curación, ocultamiento y victoria.

Vista (org.vista)

- Interfaz por consola.
- Render del mapa con colores.
- Menús interactivos y validación de comandos.

Controladores (org.controlador)

- Orquestan la interacción entre la vista y el modelo.
- Manejan flujos de juego, etapas iniciales y turnos.

Archivos (org.archivos)

- Carga de mapas y ejércitos desde archivos de configuración.

Main (org.main.Main)

- Punto de entrada del programa.

5.2. Diagrama de clases UML

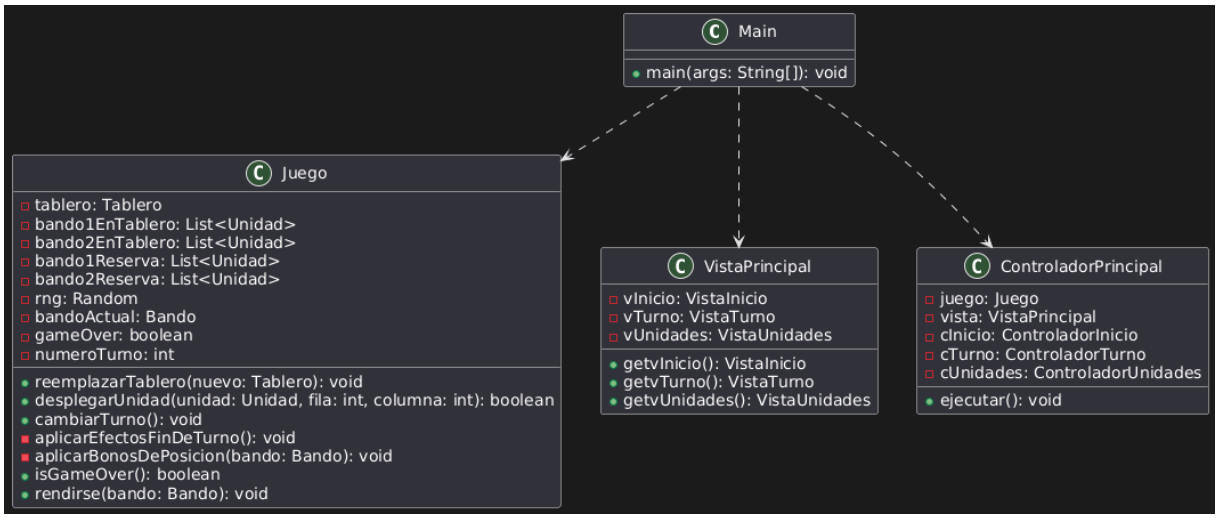
Para facilitar la comprensión del diseño del proyecto, decidimos dividir el diagrama UML en varias partes, agrupando las clases según su funcionalidad principal. De esta manera, el diagrama resulta más claro y legible. Además, optamos por omitir los métodos getters y setters de los atributos, ya que su inclusión haría el diagrama redundante sin aportar información relevante al entendimiento general del sistema.

Los diagramas se organizaron de la siguiente forma:

- UML Archivos
- UML Controlador
- UML Main
- UML Modelo
- UML Vista

Por favor, se recomienda visualizar los archivos directamente desde el repositorio del proyecto para poder hacer zoom y apreciar mejor los detalles de cada diagrama.

5.2.1. UML MAIN

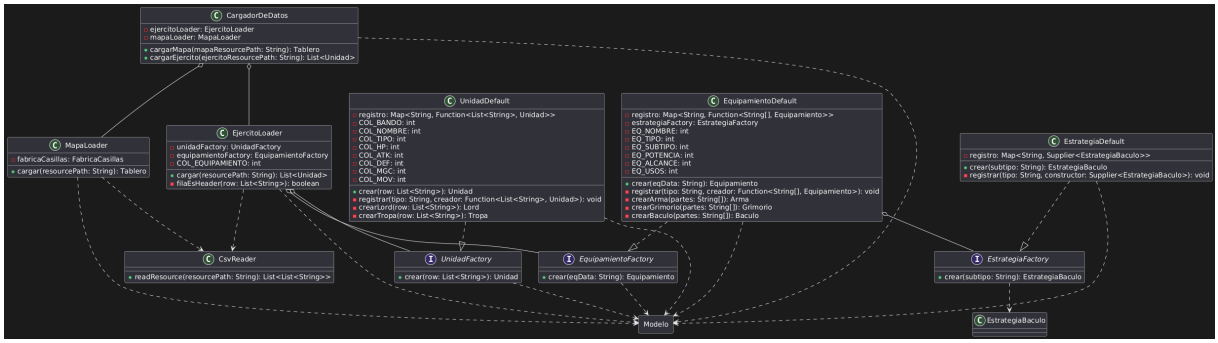


Este diagrama representa la estructura de inicializacion del juego, mostrando como la ejecución comienza y orquesta la creación de las capas principales (Modelo, Vista y Controlador).

La responsabilidad de Main es instanciar los componentes clave del patrón MVC (Modelo-Vista-Controlador) y comenzar con el flujo.

- Flujo de Inicio: dentro del método main(), se crea una instancia de Juego (el estado), VistaPrincipal (la interfaz) y luego ControladorPrincipal. Al ControladorPrincipal se les pasa las instancias y finalmente se inicia el juego llamando a ControladorPrincipal.ejecutar().
- ControladorPrincipal: compone los objetos de la capa y delega la ejecución de las fases principales (inicio y turnos), enlazando el Juego con las vistas especificas (vInicio, vTurno, vUnidades) obtenidas de VistaPrincipal.

5.2.2. UML ARCHIVOS



Este diagrama muestra las clases responsables de la lectura y construcción de los objetos del juego a partir de archivos de recurso y utilizando el patrón Factory para construir objetos de dominio.

La clase CsvReader es la capa de acceso a datos de mas bajo nivel, encargada de leer el archivo, dividir las lineas y devolver los datos sin procesar. Luego las clases EjercitoLoader y MapaLoader son las que orquestan la carga.

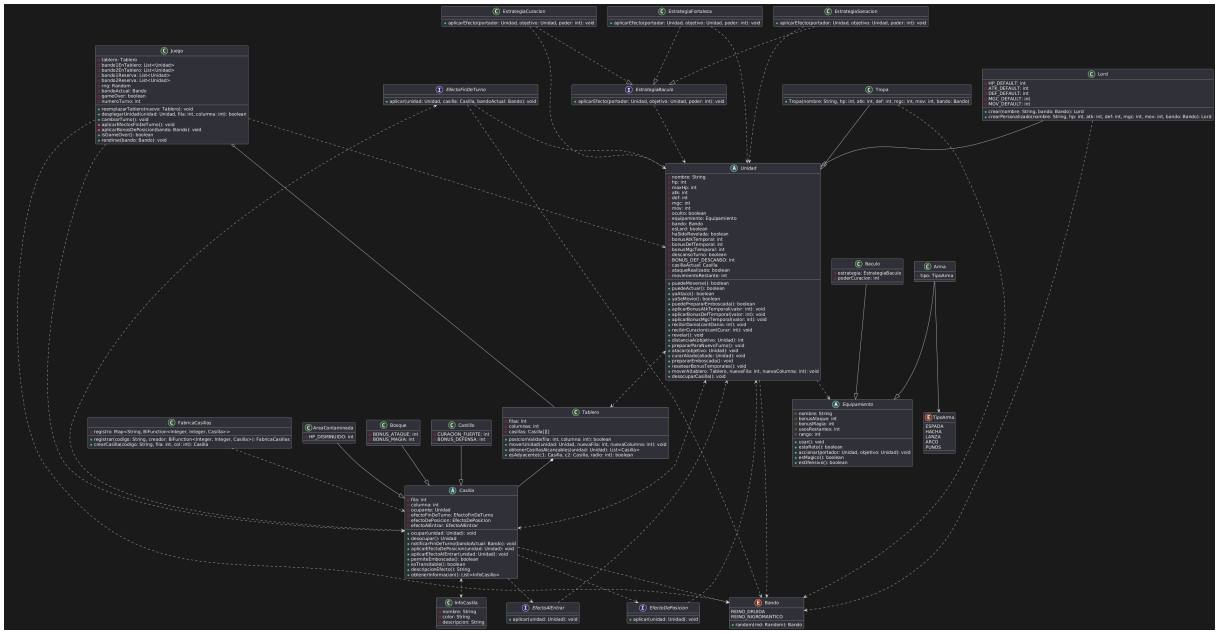
- EjercitoLoader: utiliza UnidadFactory y EquipamientoFactory para construir las unidades y su equipamiento.
- MapaLoader: utiliza FabricaCasilla (del Modelo) para construir el tablero.

Ademas, la clase CargadorDeDatos actua como una fachada que centraliza los metodos cargarMapa y cargarEjercito, simplificando su uso para el ControladorInicio.

Por otro lado, las interfaces UnidadFactory, EquipamientoFactory y EstrategiaFactory definen el contrato para la creacion de objetos del Modelo. Sus implementaciones contienen la logica para parsear filas de datos y construir los objetos especificos del dominio (como Lord, Arma, etc...).

La relacion con Modelo se hace para simplificar la representacion de multiples dependencias. Este representa clases como Unidad, Equipamiento, Tablero, Lord, etc. De esta manera, el diagrama se vuelve mas legible al evitar que numerosas lineas de dependencias crucen el diagrama para apuntar a muchas clases del dominio.

5.2.3. UML MODELO



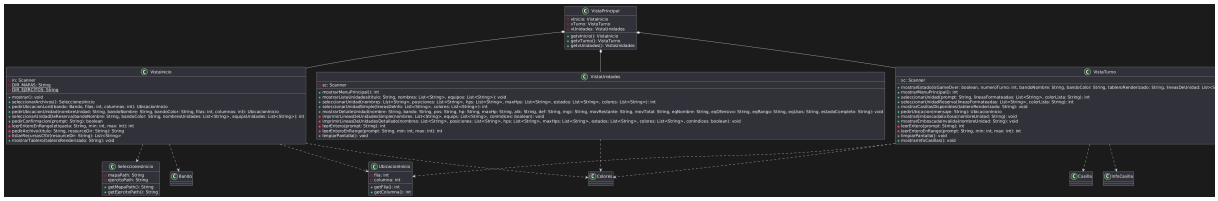
Este diagrama muestra el **Modelo** del juego, es decir, las clases que representan el estado de la partida y sus reglas básicas.

- **Juego** es la parte principal del modelo: guarda el `Tablero`, las listas de unidades de cada `Bando` (en tablero y reserva), el turno actual y si la partida terminó. A través de métodos como `desplegarUnidad`, `cambiarTurno`, `isGameOver` y `rendirse` implementa las reglas generales de victoria y avance de turno.
- **Tablero** modela la grilla de `Casilla[][]`. Concentra toda la lógica de movimiento y vecindad.
- La jerarquía de **Casilla** representa los distintos terrenos (`AreaContaminada`, `Bosque`, `Castillo`, etc.). La clase abstracta define posición, ocupante y tres tipos de efectos (`EfectoAlEntrar`, `EfectoDePosicion`, `EfectoFinDeTurno`) que se aplican sobre una `Unidad` y dependen del tipo de casilla. `InfoCasilla` sirve para exponer nombre, color y descripción del terreno a la Vista.
- Las clases **Unidad**, **Lord** y **Tropa** modelan a los personajes. `Unidad` define estadísticas, bandos, estado de turno y acciones (moverse, atacar, curar, preparar emboscadas). `Lord` y `Tropa` son clases concretas: el `Lord` se crea con métodos estáticos y actúa como líder del bando; `Tropa` es una unidad común.
- En **Equipamiento**, la clase abstracta `Equipamiento` define armas/báculos genéricos. `Arma` se asocia a un `TipoArma` y `Baculo` delega su efecto en una `EstrategiaBaculo`. Las estrategias (`EstrategiaCuracion`, `EstrategiaFortaleza`, `EstrategiaSanacion`) implementan el patrón *Strategy* para permitir distintos efectos sin modificar la clase del báculo.
- **FabricaCasillas** aplica el patrón *Factory*: mantiene un mapa de códigos a funciones creadoras y permite construir casillas concretas a partir de los códigos leídos de archivos, para agregar nuevos terrenos sin tocar la lógica de carga.

Para mantener el diagrama legible, no se incluyeron todos los tipos concretos de casilla ni las excepciones `CasillaIntransitableException`, `CasillaOcupadaException`

y `DespliegueException`: estas intervienen en la validación de movimientos y despliegues, pero en el UML hacían el gráfico demasiado denso y difícil de leer.

5.2.4. UML VISTA



Este diagrama representa las clases responsables de la interfaz del usuario, la entrada de datos y la renderizacion de la informacion.

La clase VistaPrincipal es un contenedor que agrega y gestiona las tres vistas funcionales principales, donde cada una maneja una etapa de la interfaz del usuario:

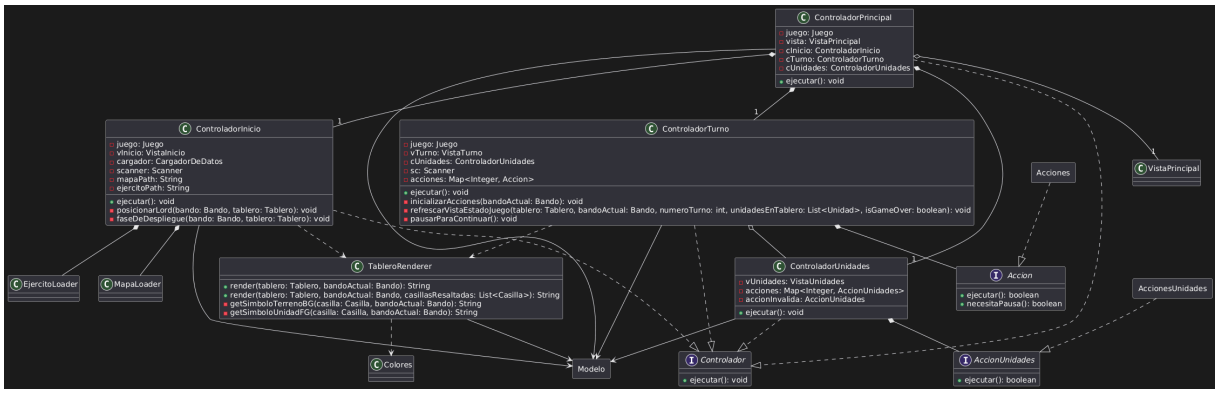
- VistaInicio: se ocupa de la configuración inicial (selección de archivos, posicionamiento de Lords) y devuelve objetos de datos estructurados como SeleccionesInicio y UbicacionInicio.
- VistaTurno: muestra el estado del juego (tablero, unidades, menú de acciones) y maneja las selecciones de turno.
- VistaUnidades: se enfoca en listar unidades y mostrar sus detalles.

Las clases auxiliares como SeleccionesInicio y UbicacionInicio son estructuras de datos utilizadas por las vistas para poder encapsular la información ingresada por el usuario (como coordenadas o rutas de archivos) y pasarla al Controlador.

Otras relaciones con otras clases:

- Colores: se utiliza por todas las vistas para aplicar formato a la consola y mejorar la legibilidad.
- Modelo (Bando, Casilla, InfoCasilla): las vistas necesitan conocer tipos del dominio (como Bando para aplicar el color correcto) y estructuras para mostrar información detallada del terreno.

5.2.5. UML CONTROLADOR

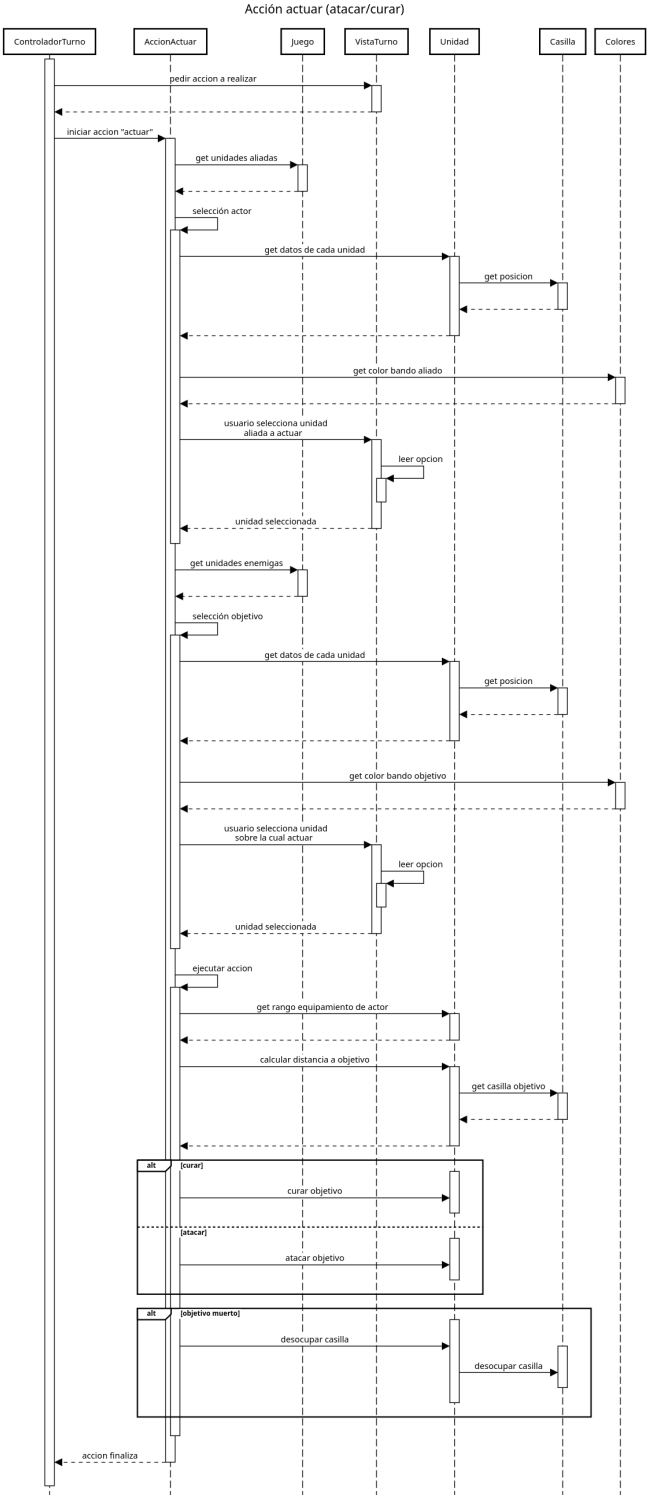


Este diagrama representa la funcionalidad del Controlador, que como se mencionó anteriormente orquesta el flujo del programa. Dado que este tiene alcance a toda parte del proyecto, se tomó la decisión de mostrar ciertas conexiones e implementaciones de forma ambigua por cuestiones de claridad, como es el caso de "Modelo", que representa de forma general a clases y abstracciones pertinentes a dicha sección del programa. De la misma forma, se omitió mostrar las partes correspondientes a la Vista, aunque información al respecto sigue presente en los atributos de los respectivos controladores.

Por otra parte, no se muestran todos los tipos de acción implementados (concretos) por ser repetitivos y hacer muy denso al gráfico, sí se mantienen sus abstracciones correspondientes. Tampoco fueron incluidas todas las clases responsables del manejo de archivos, tanto por legibilidad como porque no aportaban información relevante al diagrama.

- ControladorInicio: opera el inicio a la partida (selección de mapa y unidades, ubicación de Lords).
- ControladorTurno: maneja el flujo en cada turno, esto es cada acción seleccionada del menu y sus consecuencias.
- ControladorUnidad: maneja el submenu de unidades, que se encarga de informar en detalle sobre ellas.

5.3. Diagrama de secuencia



Este diagrama muestra el flujo de acción de las unidades, cubriendo los dos posibles escenarios: curar un aliado o atacar un enemigo.

Se optó por realizar este flujo dado que realiza uno de los recorridos más complejos, involucrando varias secciones del proyecto. A continuación una breve explicación de la secuencia:

Suponemos que el jugador eligió actuar, para cuyo caso ControladorTurno llama a

ejecutar el tipo de acción correspondiente. Para efectuarla obtenemos las posibles unidades a actuar (unidades aliadas disponibles), junto a los datos que hagan falta. El usuario seleccionará aquella que desee, y una vez hecho eso se repetirá el proceso para elegir aquella a curar o atacar, que será respectivamente aliada o enemiga.

Una vez tenemos a ambas unidades, se hacen los chequeos correspondientes sobre la validez de la acción y finalmente se lleva a cabo, para cualquiera de ambas opciones. Finaliza chequeando si la unidad murió, en cuyo caso simplemente se desocupa la casilla de la unidad derrotada.

6. Hipótesis

Durante el desarrollo del trabajo fue necesario adoptar una serie de hipótesis para resolver ambigüedades del enunciado y definir comportamientos que no estaban explícitamente especificados. Estas decisiones se tomaron buscando mantener coherencia interna, fluidez en la jugabilidad y simplicidad en la implementación, preservando siempre el equilibrio entre los dos bandos. A continuación se detallan las principales suposiciones asumidas, agrupadas según el aspecto del juego que afectan.

Hipótesis sobre el despliegue y la preparación inicial

- El archivo de ejército contiene las unidades de ambos bandos, y el sistema asigna automáticamente cada una al **Reino Druida** o al **Reino Nigromántico** según el campo correspondiente en el CSV.
- El Lord de cada bando debe posicionarse **manualmente** en una casilla válida al comenzar la partida. Se asumió que no existe una posición por defecto, ya que la elección del punto inicial influye directamente en la estrategia y el control territorial inicial.
- Las unidades en reserva solo pueden desplegarse en casillas **adyacentes al Lord**, tanto durante la fase inicial como en el transcurso de la partida. Esta restricción modela la idea de una zona de influencia del líder y evita desplegar tropas en posiciones ventajosas sin planificación previa.
- La posibilidad de desplegar en modo *oculto* se habilita únicamente en terrenos que lo permiten (como bosques), siguiendo la lógica de camuflaje natural del entorno.

Hipótesis sobre movimiento y alcance

- Se asumió que el movimiento de las unidades es posible en las **ocho direcciones** adyacentes (horizontal, vertical y diagonal), de modo que el desplazamiento no se limite a caminos ortogonales.
- Para simplificar la búsqueda de caminos, se considera que una unidad puede moverse a cualquier casilla dentro de su rango de movimiento si todas las casillas intermedias son transitables y están desocupadas.
- El **rango de ataque o curación** se determina exclusivamente por el equipamiento de la unidad (arma, grimorio o báculo), sin modificadores externos por terreno ni por habilidades especiales.
- Una unidad que realiza un ataque o curación pierde la posibilidad de moverse por el resto del turno.

Hipótesis sobre combate y efectos de terreno

- Las estadísticas de ataque (ATK) y magia (MGC) se combinan con la potencia del equipamiento utilizado, sin introducir factores aleatorios de precisión, evasión o golpes críticos.

- El daño resultante de un ataque se calcula restando la defensa del objetivo (DEF o MGC según el tipo de ataque), truncando en cero los valores negativos. Es decir, un ataque no puede curar ni generar efectos inversos.
- Los efectos de terreno se aplican automáticamente al entrar o al finalizar el turno, según su tipo. Por ejemplo, los castillos otorgan curación y bonificación defensiva, mientras que las áreas contaminadas infligen daño. Esta lógica se implementó en la notificación de fin de turno dentro del modelo.
- Las casillas que permiten emboscadas no revelan a la unidad oculta a menos que esta ataque, sea atacada o se desplace. De esta forma, el estado de ocultamiento se mantiene hasta que se produce una interacción directa.

Hipótesis sobre el sistema de turnos

- El bando inicial se decide de forma **aleatoria** al cargar los datos del ejército, evitando sesgos a favor de un jugador. Se consideró que ambos bandos tienen condiciones simétricas de inicio.
- Al comienzo de cada turno, todas las unidades del bando activo restauran su movimiento y pierden los efectos temporales aplicados en turnos previos (por ejemplo, bonus defensivos).
- Si una unidad no realiza ninguna acción ofensiva o de soporte durante su turno, obtiene un **bonus defensivo temporal** para el siguiente turno del oponente.
- La muerte de una unidad provoca la liberación inmediata de su casilla, permitiendo que otros personajes se muevan o actúen sobre ese espacio en el mismo turno.

Hipótesis sobre interacción y validaciones

- Las coordenadas ingresadas por el jugador se validan para garantizar que se encuentren dentro de los límites del tablero. Si la posición no es válida, se muestra un mensaje de error y se solicita una nueva entrada.
- Si un equipamiento agota sus usos, la unidad continúa siendo utilizable para moverse, emboscar o servir de apoyo táctico, pero no puede atacar ni curar hasta reemplazar su arma o báculo.
- El submenú de unidades tiene carácter puramente informativo: consultar estadísticas o listas no consume acciones ni modifica el estado del turno actual.
- Ante cualquier entrada inválida (opción inexistente o formato incorrecto), el sistema notifica el error y vuelve a solicitar la acción.

Hipótesis sobre la finalización del juego

- La derrota del Lord implica el fin inmediato de la partida, independientemente del estado del resto de las unidades. Este comportamiento se definió para mantener coherencia con el objetivo principal del juego: proteger al líder del bando.

- No se considera derrota si todas las demás unidades han sido eliminadas, mientras el Lord permanezca con vida, ya que este sigue representando el núcleo del ejército.
- La rendición voluntaria finaliza el juego de manera directa, sin secuencias adicionales ni fases de confirmación. Se consideró una acción definitiva y consciente del jugador.

7. Conclusiones

El desarrollo de este trabajo práctico permitió consolidar conceptos de diseño modular y orientación a objetos mediante la implementación de un sistema de juego táctico por turnos, con reglas, interacción y lógica completamente funcionales. El proyecto integró distintos componentes (modelo, vista y controlador) en una arquitectura claramente separada, lo que facilitó el mantenimiento, la extensibilidad y la comprensión del flujo interno del juego.

Aspectos técnicos y de diseño

A nivel técnico, se logró construir un sistema cohesivo que permite gestionar de forma estructurada los distintos estados del juego: inicio, despliegue, combate y finalización. La definición de clases específicas para terrenos, unidades y efectos posibilitó un alto nivel de reutilización de código y una jerarquía clara de responsabilidades. Asimismo, el uso de patrones de diseño garantizan una interacción ordenada entre los componentes del sistema.

Esto permitió obtener un producto funcional, robusto y fácilmente verificable desde la consola, ideal para entornos académicos y de prueba.

Aprendizajes y desafíos

Durante el desarrollo se enfrentaron desafíos vinculados principalmente con la gestión del flujo de turnos, la sincronización de efectos entre rondas y la representación de estados intermedios de las unidades (como el descanso, el ocultamiento o la pérdida de equipamiento). Resolver estos problemas implicó aplicar principios de encapsulamiento, abstracción y responsabilidad única en cada clase, además de una depuración iterativa a medida que se verificaban las interacciones entre los distintos módulos.

Otro aspecto relevante fue el diseño de la interfaz por consola, que requirió mantener la claridad visual y la legibilidad del tablero en todo momento. Se buscó que los mensajes fueran informativos y consistentes, de modo que el jugador pudiera comprender cada cambio de estado sin ambigüedades.

Proyección y extensiones futuras

El diseño deja abierta la posibilidad de incorporar nuevas funcionalidades sin afectar la arquitectura base. Entre las extensiones posibles se destacan:

- Incorporar una interfaz gráfica sobre la vista existente, reutilizando la lógica del modelo y los controladores actuales.
- Añadir tipos de terreno adicionales con efectos combinados (por ejemplo, pantanos mágicos o fortalezas encantadas).
- Incluir habilidades especiales de unidades y bonificaciones por afinidad de bando o tipo de arma.

- Implementar un modo multijugador remoto o cooperativo.

Estas mejoras podrían integrarse sin reescribir los componentes centrales del juego, demostrando la eficacia del diseño orientado a la extensibilidad.

Reflexión final

En síntesis, el trabajo permitió aplicar de manera práctica los principios fundamentales del paradigma orientado a objetos, desarrollando un sistema que combina interacción, estrategia y estructura. El resultado final no solo cumple con las funcionalidades propuestas, sino que también constituye una base sólida para futuras iteraciones más completas o visuales del juego.